

WinDbg Cheat Sheet

Help & Expressions	
.help command	Debugger command help
.hh command	Open command documentation
!help	List extension commands
? expr	Evaluate a MASM expression
?? expr	Evaluate a C++ expression
.formats expr	Show common numeric formats
? poi(address)	Dereference a pointer with MASM
?? sizeof(TYPE)	Return a typed object's size

Execution Control	
g [address]	Continue, optionally until address
p [count]	Step over calls
t [count]	Step into instructions
gu	Run until the current function returns
pa address	Run until address, stepping over calls
pt	Run until a return instruction
ph [count]	Run until the next branch
Ctrl+Break	Break a running live target

Registers & Frames	
r	Display registers
r reg	Display one register
r reg=value	Change a register on a live target
.frame n	Select stack frame n
.frame /r n	Select frame & display registers
dv	Display local variables in the frame

Number Prefixes	
0x	Hexadecimal (default)
0n	Decimal
0t	Octal
0y	Binary
n [radix]	Display or set radix to 8 , 10 , or 16

Target & Processor	
vertarget	Display target version & session details
!sysinfo	Display target system information
!cpuinfo [n]	CPU information for one or all processors
~n s	Select processor n
2k	Display processor 2's stack
!running -it	Running threads & stacks on all CPUs
!prcb [n]	Processor control block summary
!irq1 [n]	Saved IRQ1 for a processor
!pcr [n]	Legacy PCR summary, may be inaccurate
!idt	Interrupt descriptor table

Processes & Threads	
!process 0 0	List all active processes
!process -1 0	Display the current target process
!process EPROCESS 0x6	Threads, waits, & stacks for a process
!thread ETHREAD	Thread summary, waits, & stack
!thread -t TID	Find a thread by system TID
.process /p /r EPROCESS	Set implicit process, decode its transition PTEs, & reload UM symbols
.thread /p /r ETHREAD	Set thread register & owning process context

Objects & Handles	
!object address	Object name, type, & reference counts
!object \	Object Manager root directory
!handle HANDLE 0x3 EPROCESS	Handle & referenced object in a process
!token TOKEN	Security token contents
!fileobj FILE_OBJECT	File-object details

Breakpoints	
bl	List breakpoints
bp address	Set a resolved software breakpoint
bu module!symbol	Set an unresolved symbolic breakpoint
bm module!pattern	Set breakpoints on matching symbols
ba r size address	Break on read or write. Size is 1 , 2 , or 4 on x86, plus 8 on x64
ba w size address	Break on write. Size is 1 , 2 , or 4 on x86, plus 8 on x64
ba e 1 address	Break on execution
bc n	Clear breakpoint n
bc *	Clear all breakpoints
bd n	Disable breakpoint n
be n	Enable breakpoint n

Stacks & Disassembly	
k	Display a stack trace
kb	Stack with first three parameters
kp	Stack with full parameter information
kv	Verbose stack with frame information
kn	Stack with frame numbers
u address Lcount	Disassemble forward
ub address	Disassemble backward
uu address	Continue disassembly past memory read errors
uf module!function	Disassemble a complete function
a address	Assemble 32-bit x86 instructions into memory

Logging	
.logopen file	Start logging debugger output
.logappend file	Append to an existing log
.logclose	Close the active log
.echo text	Print text to output & log

Scheduler & Synchronization	
!ready	Threads waiting in ready queues
!running	Current & next thread per active CPU
!dpcs	Queued DPCs for each processor
!timer	Kernel timer queues
!locks	Held executive resource locks
!qlocks	Queued spin-lock state
!stacks 2	Kernel thread stacks with full details
!findthreads -i IRP	Find threads associated with an IRP
!findthreads -d DEVOBJ	Find threads associated with a device

Structures	
dt nt!_EPROCESS addr	Executive process object
dt nt!_ETHREAD addr	Executive thread object
dt nt!_KTHREAD addr	Dispatcher & scheduler thread state
dt nt!_KPRCB addr	Per-processor kernel state
dt nt!_DRIVER_OBJECT addr	Loaded driver object
dt nt!_DEVICE_OBJECT addr	Device stack object
dt nt!_IRP addr	I/O request packet
dt nt!_IO_STACK_LOCATION addr	One IRP stack location
dt nt!_OBJECT_HEADER addr	Object Manager header

KM Symbols	
!m k	List KM modules
!lmi module	Detailed module & symbol status
.reload /f driver.sys	Force symbols for one driver
sxe ld:driver.sys	Break when a driver loads
bu driver!function	Breakpoint that survives deferred load

Symbols & Modules	
.sympath	Display the symbol search path
.symfix C:\Symbols	Configure Microsoft symbols & cache
.reload module	Refresh module & defer symbol loading
.reload /f module	Force symbols for one module
ld module	Load symbols for one module
!m	List known modules & symbol status
!m l	Only modules with symbols loaded
!m e	Only modules with symbol problems
!m m pattern	Filter by module name
!n address	Find the nearest symbol
x module!pattern	Search symbol names
x /t module!pattern	Include symbol type when available

Memory & Types	
db addr Lcount	Bytes
db /c n addr	Display n byte columns
dw addr Lcount	2-byte words
dw addr Lcount	2-byte words with ASCII
dd addr Lcount	4-byte double words
dq addr Lcount	8-byte quad words
dc addr Lcount	4-byte values with ASCII
df addr Lcount	4-byte floating-point values
dd addr Lcount	8-byte floating-point values
dp addr Lcount	Pointer-sized values
da addr	ANSI string
du addr	Unicode string
dds addr Lcount	4-byte values with symbols
dqs addr Lcount	8-byte values with symbols
dps addr Lcount	Pointer-sized values with symbols
s -b range bytes	Search for bytes
s -w range value	Search for a 2-byte value
s -d range value	Search for a 4-byte value
s -q range value	Search for an 8-byte value
s -a range "text"	Search for ANSI text
s -u range "text"	Search for Unicode text
dt module!_TYPE addr	Interpret memory as a type
dt TYPE addr field	Display selected fields
dt -r1 TYPE addr	Expand one nested level
dx expression	Evaluate a typed data-model expression

Drivers & Devices	
!drvobj \Driver\Name	Driver object summary
!drvobj \Driver\Name 7	Include devices & dispatch routines
!devobj DEVOBJ	Device object & attached devices
!devstack DEVOBJ	Device stack containing the object
!devnode DEVNODE	Plug & Play device-node state
!object \Driver	Enumerate named driver objects
!object \Device	Enumerate named device objects

I/O	
!irp IRP	IRP state & I/O stack locations
!irpfind	Scan pool for allocated IRPs, can be slow
!ioctldecode IOCTL	Decode an I/O control code
!fileobj FILE_OBJECT	File object state & name
!devhandles DEVOBJ	Open handles for a device, private symbols required for full output

Context Rules	
Kernel virtual address	Normally accessible from every process context
User virtual address	Translated through the implicit process
Register context	Selected by processor, .thread , .trap , or .cxr
g	Clears temporary register contexts when execution resumes

UM Context	
.attach PID	Attach to a process
.restart	Restart a process launched by the debugger
.detach	Detach while leaving the process running
qd	Detach & quit the debugger
 	List debugged processes
[n s	Select debugger process n
~	List threads in the current process
~.	Display the current thread
~n s	Select debugger thread n
~~[TID]s	Select a thread by system TID
~* k	Display every thread's stack
!runaway 7	Thread UM, KM & elapsed times
!findstack symbol 1	Threads & frames containing a symbol

Exceptions & Event Filters	
!analyze -v	Analyze the current exception
!analyze -hang	Analyze a suspected hang
.exr -1	Display the most recent exception record
.ecxr	Select the stored exception context
.cxr address	Select a saved context record
sx	List exception & event filters
sxe code	Break when the event filter occurs
sxd code	Break only when unhandled
sxn code	Notify without breaking
sxi code	Ignore the event

UM Memory & State	
!address address	Virtual memory region information
!vprot address	Virtual memory protection
!dh address	Portable executable headers
!heap	List process heaps
!heap -x address	Find the heap block containing address
!dlls	Loaded modules from the loader lists
!gle	Current thread's last-error value
!error code	Decode a Windows error code
!exchain	x86 structured exception chain

UM Structures	
!peb	Current process environment block
!teb	Current thread environment block
!handle handle 0xf	Detailed handle information

Virtual Memory & Pool	
!vm	System virtual-memory summary
!memusage	Physical page usage summary
!address address	Kernel virtual-address region
!pte VA	PDE & PTE mapping a virtual address
!vtop PFN VA	Translate VA with a directory-base PFN
!vad VAD-Root	Display a process VAD tree
!sysptes	System PTE information
!pool address	Pool allocation containing address
!poolfind Tag 0	Scan nonpaged pool for tag, can be slow
!poolfind Tag 1	Scan paged pool for tag, can be slow
!poolused	Pool usage grouped by tag

Bug Checks & Exceptions	
!analyze -v	Verbose exception or bug-check analysis
.bugcheck	Bug-check code & four parameters
.exr -1	Most recent exception record
.trap address	Select a trap-frame register context
.cxr address	Select a context-record register context
r cr2	Faulting linear address on x86 or x64
!pte @cr2	Inspect the faulting address translation
!verifier	Driver Verifier state & statistics
!deadlock	Verifier deadlock-detection data
!blackboxnp	Captured PnP black-box data when present
!blackboxntfs	Captured NTFS black-box data when present